PROJECT REPORT

DATA ANALYTICS

THE SALES DATA JOURNEY

From raw transactions to an interactive business intelligence dashboard

Excel | MySQL | Python | Power BI

Presented by

ISHAK AHAMED N

Vinsup Skill Academy

2026

ABSTRACT

Retail organisations generate large amounts of sales data, but scattered spreadsheets make analysis difficult. This project, Retail Sales Performance Analysis: The Sales Data Journey, transforms raw retail data into meaningful business insights using Microsoft Excel, MySQL, Python, and Power BI. Excel was used to clean nine source files, remove duplicates, handle missing values, standardise column names, and calculate total sales.

The cleaned data was stored in MySQL using primary and foreign keys to maintain data integrity. Python, using pandas and SQLAlchemy, supported data extraction and exploratory analysis, while Matplotlib helped visualise sales patterns and trends.

Customer purchasing behaviour was analysed using Recency, Frequency and Monetary (RFM) analysis. StandardScaler and K-Means clustering were used to group customers into Loyal, New, and At-Risk segments. These segments were stored in MySQL for further analysis.

Power BI was connected to MySQL to create an interactive dashboard featuring Total Sales, Total Orders, and Average Sales, along with filters for year, category, and brand. The analysis identified total sales of ₹8,579K across 1,615 orders, with New York as the top-performing market, Mountain Bikes as the leading category, and Trek as the leading brand. Sales peaked in April.

Based on these findings, the project recommends focusing on high-performing markets, maintaining sufficient stock of popular products, preparing for seasonal demand, and developing customer loyalty programmes. Overall, the project demonstrates how data analytics can transform raw retail data into actionable insights and support better business decisions.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	ii
	LIST OF FIGURES	iv
	LIST OF TABLES	v
1	**INTRODUCTION**	**1**
1.1	OVERVIEW OF RETAIL ANALYTICS	1
1.2	PROBLEM STATEMENT	2
1.3	WHAT IS SALES PERFORMANCE ANALYSIS?	2
1.4	OBJECTIVES AND SCOPE	3
2	**DATA ARCHITECTURE AND PREPROCESSING**	**4**
2.1	SYSTEM ARCHITECTURE	4
2.2	PHASE 1: DATA CLEANING (MICROSOFT EXCEL)	5
2.3	PHASE 2: RELATIONAL DATABASE MANAGEMENT (MYSQL)	7
2.4	DATABASE TROUBLESHOOTING	9
3	**EXPLORATORY DATA ANALYSIS**	**10**
3.1	PYTHON DATA EXTRACTION	10
3.2	EXPLORATORY DATA ANALYSIS (EDA)	11
3.3	THE RFM CUSTOMER SEGMENTATION MODEL	12
3.4	K-MEANS CLUSTERING	14
4	**BUSINESS INTELLIGENCE & VISUALIZATION**	**16**
4.1	POWER BI INTEGRATION	16
4.2	DASHBOARD	17
4.3	DAX MEASURES	18
5	**RESULTS, RECOMMENDATIONS, AND CONCLUSION**	**20**
5.1	KEY FINDINGS & METRICS	20
5.2	BUSINESS RECOMMENDATIONS FOR STORE MANAGERS	23
5.3	CONCLUSION & FUTURE ENHANCEMENTS	24

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
2.2	System Architecture from Raw Data to Dashboard	5
2.3	Entity-Relationship Diagram of the Nine-Table Database	9
3.1	Histogram of Product List Prices	12
4.1	Power BI Data Model with Table Relationships	17
4.2	Main Dashboard	18

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
2.1	Summary of Data Cleaning Operations	7
2.2	Tables, Primary Keys and Foreign Keys	7
3.1	RFM Metrics, Formulas and Interpretation	13
3.2	Customer Segment Profiles	15
4.1	DAX Measures and KPI Cards	19
5.1	Headline Financial Metrics	20
5.2	Revenue by State	21
5.3	Revenue by Brand	22
5.5	Monthly Sales at Peak and Trough	22
5.6	Highest-Spending Customers	23
5.7	Summary of Findings and Recommendations	24

CHAPTER 1
INTRODUCTION

Retail transactions record what was sold, to whom, by which employee, at what price and when. Across months and years, these records show business performance, but only when they are organised, validated and analysed.

This project transforms fragmented retail data into an interactive business intelligence dashboard. Excel handles cleaning and feature engineering, MySQL provides relational storage and integrity, Python supports exploratory analysis, and Power BI presents the results.

1.1 OVERVIEW OF RETAIL ANALYTICS

Retail analytics processes and interprets operational data to support decisions about sales, inventory, customer behaviour, pricing, staffing and marketing. Managerial experience remains useful, but a database can compare transactions across all stores, customers and periods at a scale that manual observation cannot.

The retail market is characterised by thin margins, seasonal demand and customers who can compare prices and alternatives within seconds. In such an environment, a retailer that understands its sales patterns holds a measurable advantage. The retailer knows which products to stock before demand arrives, which stores deserve additional resources, which customers are drifting away and when to run promotions. A retailer without that understanding discovers its mistakes only after they appear as lost sales, unsold inventory or departed customers.

1.2 PROBLEM STATEMENT

Many retailers store customer, order, product and staff data in separate spreadsheets. Differences in structure and data quality make these files difficult to analyse together.

The main issues are disconnected files, inconsistent headers and dates, missing or duplicated values, invalid references, limited customer insight and static reports that cannot be filtered interactively.

These issues limit sales tracking, customer understanding and growth planning. This project addresses them through a repeatable workflow for cleaning, integrating, analysing and visualising retail data.

1.3 WHAT IS SALES PERFORMANCE ANALYSIS?

Sales Performance Analysis examines how revenue is generated and where improvement is possible. This project organises analysis around four questions linked to the database tables.

1.3.1 Who Sold? (Staff)

The staffs table links each employee to a store and manager, supporting staff-level sales comparisons and coaching decisions.

1.3.2 What Sold? (Products)

The products, categories, brands, order_items and stocks tables show best-selling products and compare demand with available stock.

1.3.3 Where Sold? (Stores)

The stores, orders and location fields identify high-performing states, cities and stores and highlight markets for improvement.

1.3.4 Who Bought It? (Customers)

The customers, orders and customer_segments tables identify top spenders and Loyal, New and At-Risk groups.

1.4 OBJECTIVES AND SCOPE

The primary objective of the project is to design and implement an end-to-end analytical pipeline that transforms raw retail transactions into actionable business intelligence. The specific objectives are listed below.

1. To integrate the disparate datasets into a relational database by cleaning them, defining nine related tables and enforcing referential integrity through primary and foreign keys.
2. To segment customers using RFM analysis and K-Means clustering, so that Loyal, New and At-Risk customers can be distinguished and treated differently.
3. To track regional sales and optimise store management by measuring revenue across states, stores, categories, brands and months.
4. To build an interactive Power BI pipeline, connected directly to the database, that provides slicers, KPI cards and visualisations for managers who do not have technical training.

1.4.1 Scope of the Project

The project covers the historical sales data for the years 2016 to 2018. It includes data cleaning, database design, exploratory analysis, customer segmentation, dashboard development and business recommendations. The analysis is descriptive and diagnostic: it explains what happened and why it matters. Predictive forecasting and real-time cloud synchronisation lie outside the present scope and are proposed as future enhancements in Chapter 5.

CHAPTER 2
DATA ARCHITECTURE AND PREPROCESSING

Reliable analysis begins with reliable data. This chapter covers source organisation, Excel cleaning, MySQL loading and the resolution of integrity errors that could otherwise affect every later chart and model.

2.1 SYSTEM ARCHITECTURE

The pipeline moves from raw files to Excel cleaning, MySQL storage, Python analysis and Power BI visualisation. MySQL is the central source: Python reads the data and writes customer segments back, while Power BI reads the same database.

2.1.1 Organisation of the Source Data

The source data is organised into nine files, which fall naturally into three groups, as the diagram below shows. Reference data describes the stable entities of the business: the customers who buy, the staff who sell and the stores where sales take place. Transactional data records the events: orders, the individual items within each order, and the stock held at each store. Product data describes what is sold: the products themselves, the categories into which they are grouped and the brands that manufacture them.

2.1.2 End-to-End Pipeline

The complete flow of the project can be summarised in five stages.

- **Extraction.** Nine raw files are collected from the source system.
- **Cleaning (Excel).** Duplicates are removed, headers and dates are standardised, missing values are filled and the total_price feature is engineered.
- **Storage (MySQL).** The cleaned files are imported into nine related tables protected by keys and constraints.

- **Analysis (Python).** The tables are read into dataframes for exploratory analysis, RFM calculation and K-Means clustering.
- **Visualisation (Power BI).** The database is connected to a dashboard with slicers and KPI cards.

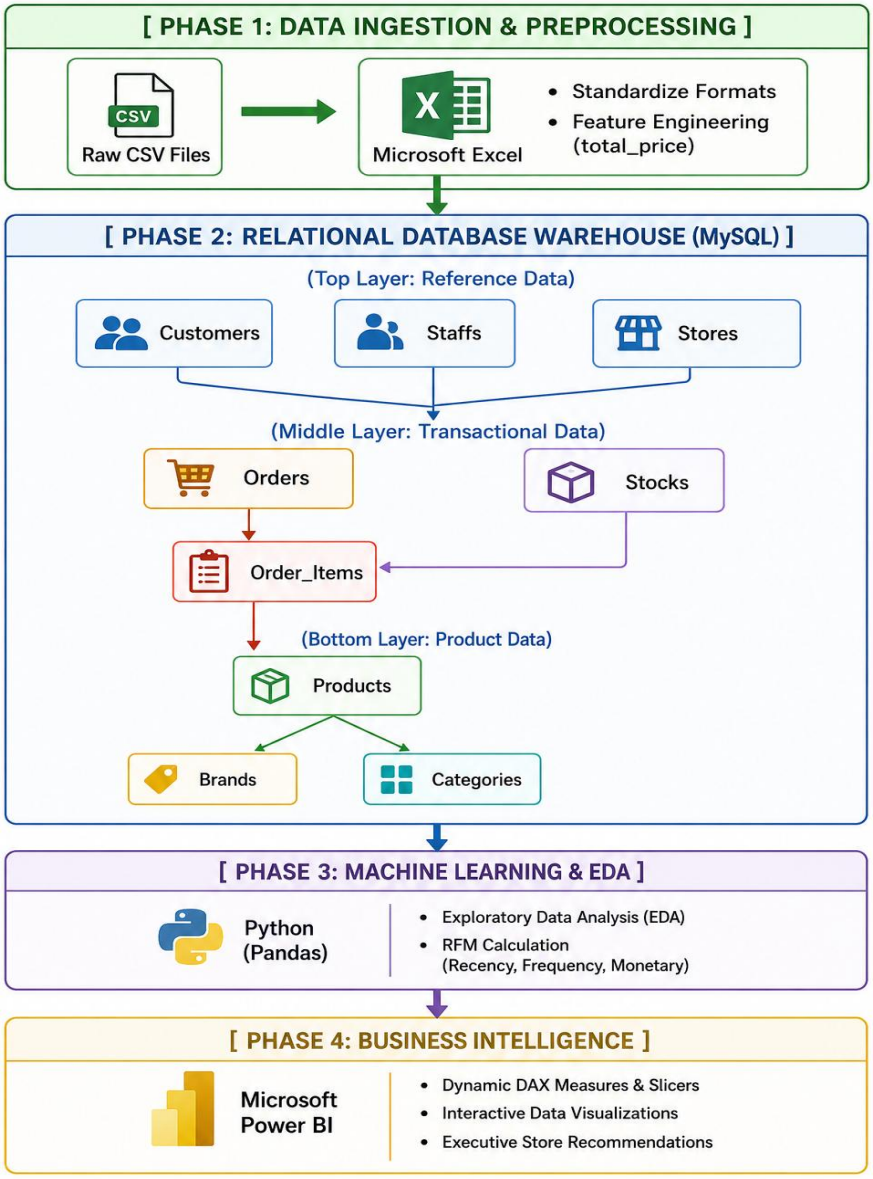


Figure 2.2 SYSTEM ARCHITECTURE FROM RAW DATA TO DASHBOARD

2.2 PHASE 1: DATA CLEANING (MICROSOFT EXCEL)

Excel was used to inspect and correct source files before database import.

2.2.1 Removal of Duplicates

Duplicates inflate counts and revenue and can violate primary keys. They were removed using each file's identifier, such as customer_id, order_id or product_id, leaving one record per entity.

2.2.2 Standardising Headers to snake_case

Raw headers contained spaces, mixed capitalisation and inconsistent wording. They were standardised to snake_case (for example, order_date, list_price and customer_id), making SQL queries, Python code and joins more consistent.

2.2.3 Formatting Date

All date fields (order_date, required_date and shipped_date) were converted to ISO 8601 (YYYY-MM-DD). This avoids regional ambiguity, imports cleanly into MySQL and supports reliable year- and month-based dashboard filters.

2.2.4 Handling Missing Values

Missing phone numbers and ZIP codes in customer and store files were filled with "NA" to retain records and make gaps explicit. This placeholder later required correction in one database column, as described in Section 2.4.

2.2.5 Feature Engineering of total_price

Feature engineering creates useful variables from existing fields. Because order_items stores list_price, quantity and discount but not the final line value, total_price was added using:

```
total_price = (list_price * quantity) - discount
```

The formula calculates net line revenue. This column drives Power BI Total Sales, the RFM Monetary value and all revenue comparisons, ensuring a consistent revenue definition across tools.

Table 2.1 SUMMARY OF DATA CLEANING OPERATIONS

Operation	Rule Applied	Reason
Duplicate removal	One row per natural identifier	Prevents inflated counts and key violations
Header standardisation	All headers in snake_case	Consistent joins and error-free queries
Date formatting	ISO 8601 (YYYY-MM-DD)	Unambiguous, MySQL-compatible dates
Missing values	Phone numbers and ZIP codes set to "NA"	Explicit, consistent treatment of gaps
Feature engineering	total_price = (list_price * quantity) - discount	Single definition of revenue

2.3 PHASE 2: RELATIONAL DATABASE MANAGEMENT (MYSQL)

2.3.1 Why a Relational Database

Unlike separate spreadsheets, a relational database links related tables and enforces valid relationships. MySQL was selected for its availability and compatibility with SQLAlchemy and Power BI. Customer details are stored once and referenced by customer_id, reducing repetition.

2.3.2 The Nine Tables

The nine cleaned files became nine tables, created in dependency order so parent tables existed before child tables.

Table 2.2 TABLES, PRIMARY KEYS AND FOREIGN KEYS

Table	Primary Key	Foreign Keys
brands	brand_id	None
categories	category_id	None
customers	customer_id	None
stores	store_id	None
staffs	staff_id	store_id (stores), manager_id (staffs)

Table	Primary Key	Foreign Keys
products	product_id	brand_id (brands), category_id (categories)
orders	order_id	customer_id (customers), store_id (stores), staff_id (staffs)
order_items	order_id and item_id	order_id (orders), product_id (products)
stocks	store_id and product_id	store_id (stores), product_id (products)

2.3.3 Primary Keys

A primary key uniquely identifies each row and cannot be null or repeated. Entity tables use single identifiers; order_items uses (order_id, item_id), while stocks uses (store_id, product_id).

2.3.4 Foreign Keys

Foreign keys enforce links between tables. Orders reference customers, stores and staff; products reference brands and categories; staffs.manager_id references another staff_id to represent the reporting hierarchy.

Foreign keys reject invalid references, protecting downstream calculations and exposing the import problems described next.

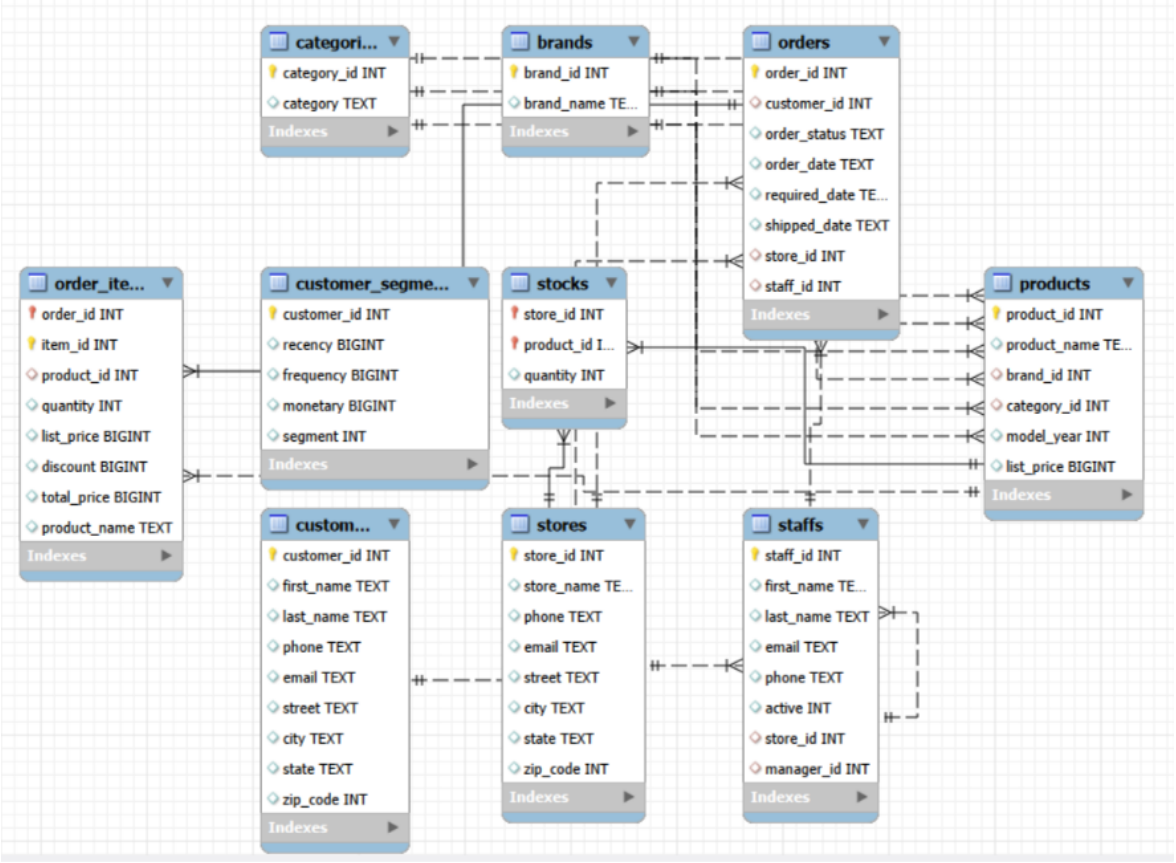


Figure 2.3 ENTITY-RELATIONSHIP DIAGRAM OF THE NINE-TABLE DATABASE

2.4 DATABASE TROUBLESHOOTING

Two import errors occurred during database construction. Both arose from data issues and demonstrated how integrity constraints help identify and prevent invalid records.

CHAPTER 3
EXPLORATORY DATA ANALYSIS

With the database prepared, Python is used to extract and profile the data, perform exploratory analysis, and segment customers with RFM and K-Means. Its libraries support database connectivity, statistics, visualization.

3.1 PYTHON DATA EXTRACTION

3.1.1 Connecting to MySQL

Two libraries provide the connection. SQLAlchemy is a database toolkit that manages the connection to MySQL through an engine object, hiding the differences between database systems. pandas is the principal data analysis library, and it provides the DataFrame, a two-dimensional table of labelled columns that resembles a spreadsheet but can be processed programmatically. The function pandas.read_sql passes a query to the engine and returns the result as a dataframe.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as seabornInstance
from sqlalchemy import create_engine
from sqlalchemy.engine import URL
DB_CONFIG = {
    'drivername': 'mysql+pymysql',
    'host': 'localhost',
    'database': 'retail',
    'username': 'root',
    'password': 'YOUR_DB_PASSWORD'
}
connection_url = URL.create(**DB_CONFIG)
engine = create_engine(connection_url)
orders_df = pd.read_sql('SELECT * FROM orders', engine)
```

```
    order_items_df = pd.read_sql('SELECT * FROM
order_items', engine)
    customers_df = pd.read_sql('SELECT * FROM customers',
engine)
    print("Data loaded successfully!")
    print("Orders shape:", orders_df.shape)
    print("Order Items shape:", order_items_df.shape)
    print("Customers shape:", customers_df.shape)
```

3.1.2 Merging the Tables

The dataframes are joined on keys: orders to order_items by order_id, customers by customer_id and products by product_id. The resulting order-line dataset combines customer, product and date information for repeatable analysis.

3.2 EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) uses summaries and visualisations to understand data structure, detect anomalies and form hypotheses before modelling.

3.2.1 Initial Data Profiling

The info() and describe() methods checked data types and summary statistics; isnull().sum() checked for remaining gaps. Matplotlib-based plots explored numerical distributions and relationships between price and quantity.

3.2.2 List Price Histogram

A Matplotlib histogram groups list_price values into bins to show the price distribution.

```
    print(orders_df.info())
    print(order_items_df.describe())
    print(orders_df['order_status'].value_counts())
    plt.hist(order_items_df['list_price'])
    plt.title("List Price Chart")
    plt.show()
```

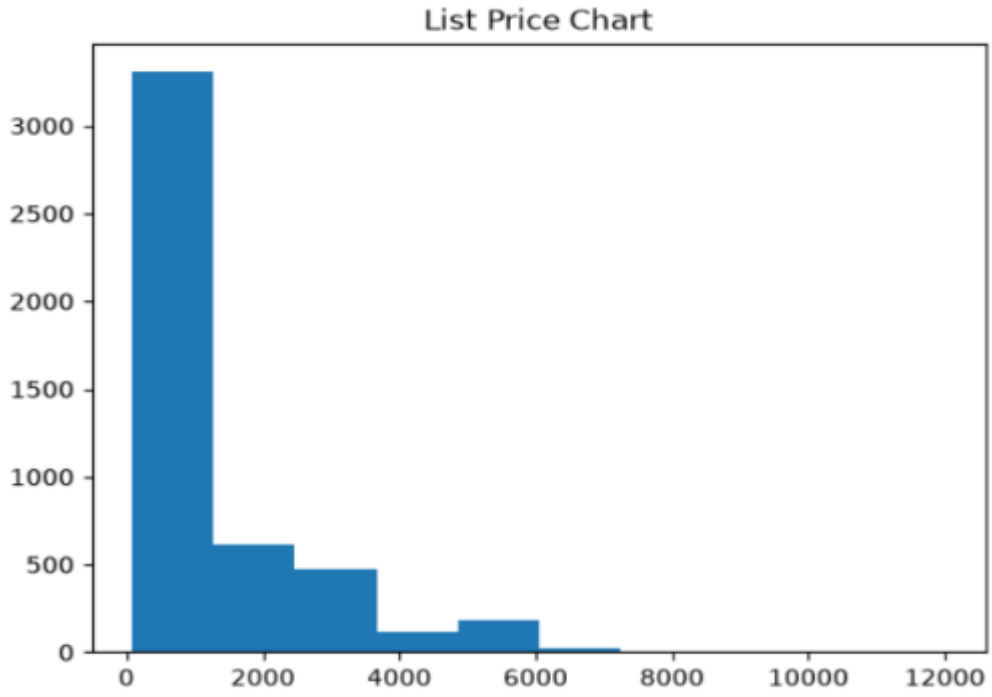


Figure 3.1 HISTOGRAM OF PRODUCT LIST PRICES

3.3 THE RFM CUSTOMER SEGMENTATION MODEL

3.3.1 Theoretical Background

RFM analysis measures customer value from transaction history using three dimensions:

- **Recency (R).** How long ago the customer last purchased. A smaller value is better, because a recent customer is still engaged with the business.
- **Frequency (F).** How many distinct orders the customer has placed. A larger value indicates habit and loyalty.
- **Monetary (M).** How much the customer has spent in total. A larger value indicates greater importance to revenue.

RFM measures complement one another: a high-spending customer who has not purchased recently may need a win-back offer, while a recent one-time buyer is an opportunity for repeat-purchase marketing.

3.3.2 Practical Implementation

The RFM table is built by grouping the merged dataframe by customer_id and applying one aggregation to each dimension.

- **Recency** is calculated as (last_date - order_date.max()).days. The reference date, last_date, is the latest order date in the dataset. For each customer, the latest purchase date is subtracted from it and the difference is expressed in days.
- **Frequency** is calculated with nunique on order_id, which counts distinct orders. The use of nunique matters because a customer's order may have many lines, and counting rows would exaggerate frequency.
- **Monetary** is calculated with sum on total_price, which adds the engineered revenue column over all of the customer's order lines.

```
orders_df['order_date'] =
pd.to_datetime(orders_df['order_date'])

df = pd.merge(orders_df, order_items_df, on='order_id')

last_date = df['order_date'].max()

rfm = df.groupby('customer_id').agg({

    'order_date': lambda x: (last_date - x.max()).days,

    'order_id': 'nunique',

    'total_price': 'sum'

})

rfm.columns = ['recency', 'frequency', 'monetary']

rfm = rfm.reset_index()

print(rfm.head())
```

Table 3.1 RFM METRICS, FORMULAS AND INTERPRETATION

Metric	Calculation	Interpretation
Recency	(last_date - order_date.max()).days	Lower value means more recent purchase

Metric	Calculation	Interpretation
Frequency	nunique of order_id	Higher value means more repeat buying
Monetary	sum of total_price	Higher value means greater revenue contribution

3.4 K-MEANS CLUSTERING

3.4.1 Why Scaling Is Necessary

K-Means uses distances, but RFM variables have different scales: days, order counts and currency. StandardScaler converts each feature to mean zero and standard deviation one so no variable dominates the distance calculation.

3.4.2 The K-Means Algorithm

K-Means is an unsupervised algorithm that assigns customers to the nearest centroid and repeatedly recalculates centroids until assignments stabilise. Three clusters were used for actionable segments, with random_state=42 for reproducibility.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
x = rfm[['recency', 'frequency', 'monetary']]
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
model = KMeans(n_clusters=3, random_state=42)
rfm['segment'] = model.fit_predict(x_scaled)
print(rfm.head())
```

3.4.3 Interpreting the Three Segments

Cluster labels were interpreted by comparing each cluster's average recency, frequency and monetary value.

- **Loyal.** High spend, frequent purchases and recent activity. These customers are the core of the business.

- **New.** Recent first-time buyers whose frequency is still low. They have not yet established a pattern and represent the greatest opportunity for conversion into loyal customers.
- **At-Risk.** High historical spend, but high recency, meaning that they have not purchased for a long time. They were once valuable, and their inactivity is a warning.

Table 3.2 CUSTOMER SEGMENT PROFILES

Segment	Recency	Frequency	Monetary	Recommended Action
Loyal	Low (recent)	High	High	Reward and retain
New	Low (recent)	Low	Low to moderate	Nurture and convert
At-Risk	High (inactive)	Moderate	High (historical)	Win back with targeted offers

3.4.4 Saving the Segments Back to SQL

The labelled clusters were written back to MySQL using to_sql, creating customer_segments for use by Power BI.

```
rfm.to_sql('customer_segments', con=engine,
if_exists='replace', index=False)
```

This closes the pipeline: Python reads from MySQL, adds segment labels and writes them back so Power BI can relate and filter customers by segment.

CHAPTER 4
BUSINESS INTELLIGENCE & VISUALIZATION

Store managers need clear insights without having to use SQL or Python. This chapter explains how Power BI presents the analysis in an interactive dashboard.

4.1 POWER BI INTEGRATION

4.1.1 Why Power BI

Power BI connects to data sources, models table relationships, calculates DAX measures and publishes interactive reports. Its MySQL connector keeps the dashboard aligned with the same database used by Python.

4.1.2 Connecting to the MySQL Database

The connection was established in the following steps.

1. The MySQL connector was installed on the machine running Power BI Desktop.
2. In Power BI Desktop, Get Data was selected and the MySQL database option was chosen.
3. The server address and the database name were entered, followed by the user credentials.
4. The navigator window listed the available tables, and the nine core tables and the customer_segments table were selected and loaded.
5. In Model view, the relationships created by the foreign keys were verified so that filters flow correctly from dimension tables to the order and order item data. For example, a filter on brand propagates through products to order_items.

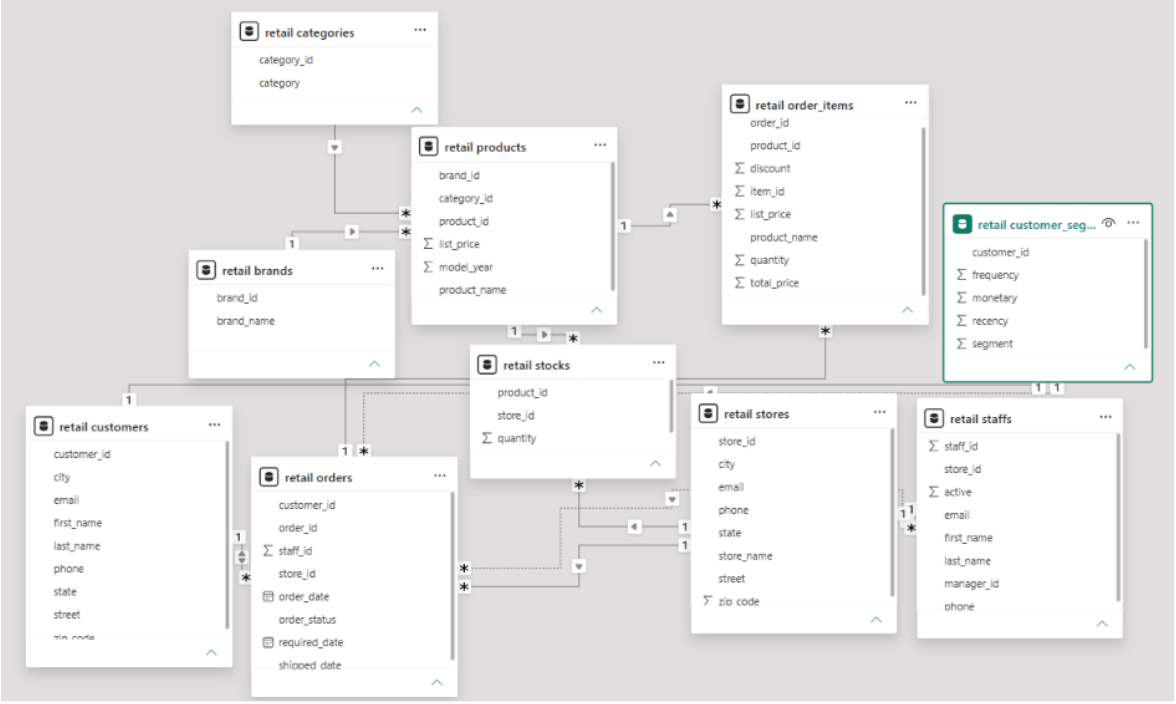


Figure 4.1 POWER BI DATA MODEL WITH TABLE RELATIONSHIPS

4.2 DASHBOARD

Dashboard design affects how quickly users find insights. The dashboard uses dark purple and pink to provide contrast and a clear visual hierarchy.

- High contrast. Pink and white KPI values stand out against the dark purple background.
- Readability. The consistent dark theme supports clear viewing.
- Visual hierarchy. Pink highlights key figures and leading bars; purple supports secondary elements.
- Consistency. Aligned visuals, a unified palette and spacing keep the dashboard professional.

KPI cards summarise performance at the top, slicers sit at the side, and the main area displays geography, product, brand and time trends.

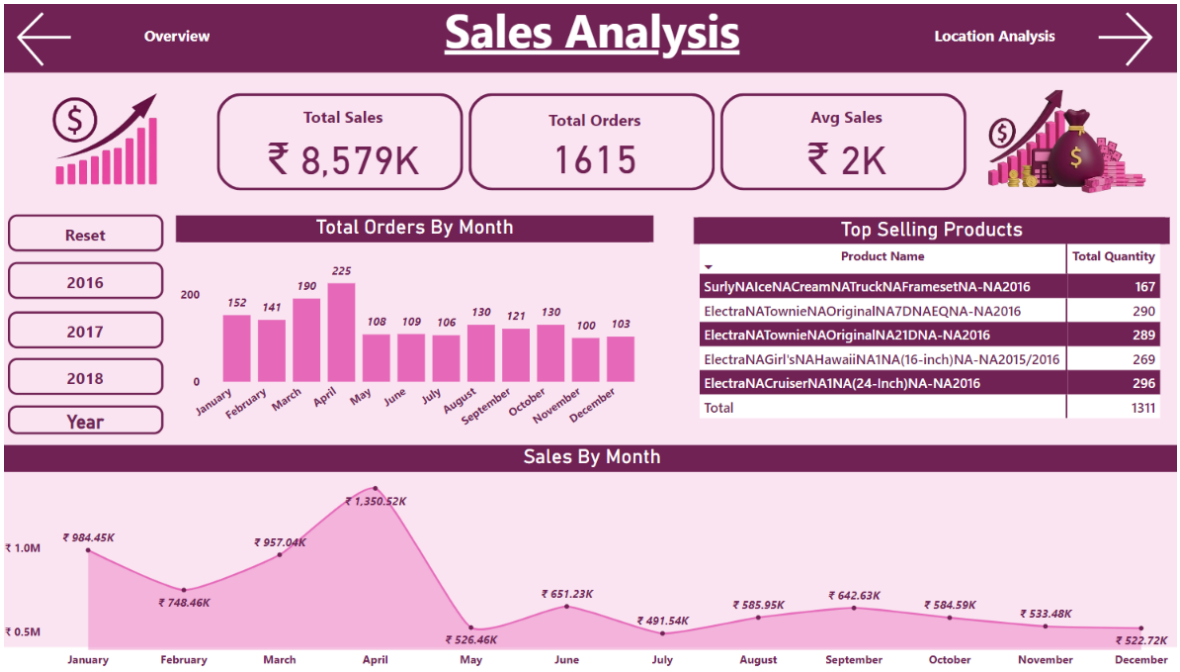


Figure 4.2 MAIN DASHBOARD

4.3 DAX MEASURES

4.3.1 Dynamic Slicers

Slicers are on-screen filters that allow the viewer to restrict every visual on the page to a selected subset of the data. Three slicers were implemented.

- **Year (2016 to 2018).** Restricts the dashboard to a single year or a combination of years, allowing year-on-year comparison.
- **Category.** Restricts the dashboard to one or more product categories, such as Mountain Bikes.
- **Brand.** Restricts the dashboard to selected brands, such as Trek, Electra or Surly.

Because the slicers are connected to the data model, a selection updates all KPI cards and charts simultaneously. A manager can, for example, select 2017 and the Trek brand and immediately see the sales of that brand by state and by month.

4.3.2 DAX Measures and KPI Cards

DAX (Data Analysis Expressions) is the formula language of Power BI. A measure is a calculation that is evaluated according to the current filters, which is why the same measure shows different values when a slicer changes. Three measures were created and displayed as KPI cards.

```
Total Sales  = SUM ( order_items[total_price] )
Total Orders = DISTINCTCOUNT ( orders[order_id] )
Average Sales = AVERAGE ( order_items[total_price] )
```

Table 4.1 DAX MEASURES AND KPI CARDS

KPI Card	DAX Logic	Business Meaning
Total Sales	Sum of total_price	Revenue earned in the selected period
Total Orders	Distinct count of order_id	Number of separate transactions
Average Sales	Average of total_price	Typical value of a sales record

4.3.3 Geographic and Product Visuals

The dashboard includes sales by state, category, brand, product and month. The customer_segments table supports filtering by Loyal, New and At-Risk customers, and a table highlights the highest-spending customers.

CHAPTER 5
RESULTS, RECOMMENDATIONS, AND CONCLUSION

This chapter summarises dashboard results, interprets their business meaning, and presents recommendations, limitations and future enhancements.

5.1 KEY FINDINGS & METRICS

5.1.1 Financial Overview

The dashboard shows that total historical revenue reached ₹8,579K across 1,615 orders, with an Average Order Value (AOV) of ₹2K. Revenue of this magnitude, earned over the period from 2016 to 2018, establishes the scale of the business and the baseline against which every other figure should be read. The modest AOV is informative in its own right. It indicates that the typical sale is of moderate value and that total revenue is built through a large number of transactions and not through a small number of very large ones. For the retailer, this means that growth depends on increasing the number of orders, through traffic, repeat purchases and retention, as much as on increasing the value of each order through upselling and bundling.

Table 5.1 HEADLINE FINANCIAL METRICS

Metric	Value
Total Sales	₹8,579K
Total Orders	1,615
Average Order Value (AOV)	₹2K
Period Covered	2016 to 2018

5.1.2 Geographic Dominance

Location analysis shows that New York (NY) is the primary market, generating ₹1,156.8K. It heavily outperforms California (CA) at ₹517.8K and Texas (TX) at ₹237.8K. New York's revenue is more than twice that of California and nearly five times that of Texas. The gap is too large to be

explained by chance variation, and it suggests that New York combines a larger customer base, stronger store performance and higher purchasing activity.

Table 5.2 REVENUE BY STATE

State	Revenue (₹K)	Relative to New York
New York (NY)	1,156.8	Baseline
California (CA)	517.8	About 45% of NY
Texas (TX)	237.8	About 21% of NY

Strong markets need reliable stock and staffing. In weaker markets such as Texas, managers should use the dashboard to check customer numbers, spending and store execution before investing in targeted improvements.

5.1.3 Product Performance

Product performance was examined at three levels: category, brand and individual product.

Category. Mountain Bikes generated ₹3,031K, which is the leading category and roughly 35 percent of total revenue. The dominance of this category is consistent with the right-skewed price distribution seen in the exploratory analysis, since mountain bikes sit at the higher price points and sell in considerable volume.

Brand. Trek is the leading brand at ₹5,129K, followed by Electra (₹1,344K) and Surly (₹1,063K). Trek alone accounts for about 60 percent of total revenue and earns nearly four times the revenue of Electra, which indicates a heavy dependence on a single supplier. This is a strength while Trek demand and supply are stable, and a risk if either is disrupted.

Table 5.3 REVENUE BY BRAND

Rank	Brand	Revenue (₹K)
1	Trek	5,129
2	Electra	1,344

Rank	Brand	Revenue (₹K)
3	Surly	1,063

Individual products. The top-moving products by units sold are the Electra Cruiser 1 with 296 units and the Electra Townie Original 7D with 290 units. An important contrast follows. Electra is second in revenue but holds the top two positions by units. Its products are therefore sold in high volume at lower unit prices, whereas Trek earns its position through higher-priced models. The two brands play different roles: Electra drives traffic and unit volume, and Trek drives revenue.

5.1.4 Seasonal Trends

Sales exhibit a pronounced seasonal pattern. There is a massive spike in the spring, peaking in April at ₹1,350.52K. This is followed by sharp declines in the mid-summer and winter months, with July falling to ₹491.54K and December to ₹522.72K. April revenue is nearly three times that of July and more than two and a half times that of December.

Table 5.5 MONTHLY SALES AT PEAK AND TROUGH

Month	Sales (₹K)	Observation
April	1,350.52	Seasonal peak
July	491.54	Mid-summer decline
December	522.72	Winter decline

The spring peak is consistent with customers preparing for the cycling season. This pattern informs inventory, staffing and marketing plans in Section 5.2.

5.1.5 Top Customers

The highest-spending customers are Pamelia (₹38K) and Lyndsey (₹36K). Although these amounts are small compared with total revenue, they are many times the average order value, and they show the scale of what a single highly engaged customer can contribute. Together with the RFM segmentation, the

list of top customers allows the business to identify the individuals who most deserve recognition and to check whether any of them have drifted into the At-Risk segment.

Table 5.6 HIGHEST-SPENDING CUSTOMERS

Customer	Total Spend
Pamelia	₹38K
Lyndsey	₹36K

5.2 BUSINESS RECOMMENDATIONS FOR STORE MANAGERS

Each recommendation below follows from a specific finding and is stated so that a store manager can act on it directly.

5.2.1 Focus on New York

Finding. New York generates ₹1,156.8K, well ahead of California and Texas.

Action. Replenish New York stores first and assign experienced staff to busy locations. Investigate weaker markets before committing additional investment.

5.2.2 Stock Up on Mountain Bikes and Trek

Finding. Mountain Bikes contribute ₹3,031K and Trek contributes ₹5,129K.

Action. Monitor the stocks table against sales and reorder Mountain Bikes and Trek early. Promote these products while developing Electra and Surly to reduce reliance on one brand.

5.2.3 Prepare for Spring Rushes

Finding. Sales peak at ₹1,350.52K in April and fall to ₹491.54K in July and ₹522.72K in December.

Action. Build inventory and schedule staff before April. Use promotions during quieter summer and winter months to stimulate demand and reduce excess stock.

5.2.4 Reward Top Buyers

Finding. A small number of customers, led by Pamelia and Lyndsey, spend far more than the average, and RFM analysis identifies those who are Loyal and those who are At-Risk.

Action. Reward Loyal customers, send win-back offers to At-Risk customers and encourage New customers to make a second purchase.

Table 5.7 SUMMARY OF FINDINGS AND RECOMMENDATIONS

Finding	Recommendation
New York leads at ₹1,156.8K	Prioritise stock and top sales staff in NY
Mountain Bikes ₹3,031K; Trek ₹5,129K	Prevent stockouts and feature prominently
April peak ₹1,350.52K	Plan staffing and marketing for spring
High-value customers drive revenue	VIP loyalty programme built on RFM

5.3 CONCLUSION & FUTURE ENHANCEMENTS

5.3.1 Conclusion

The project converts fragmented spreadsheets into decision support through four stages. Excel standardised records, dates, missing values and total_price; MySQL organised nine related tables and resolved Errors 1170 and 1452; Python performed EDA and RFM/K-Means segmentation into Loyal, New and At-Risk groups; and Power BI delivered an interactive dashboard with slicers and KPI cards.

The results show ₹8,579K revenue from 1,615 orders, concentrated in New York, Mountain Bikes and Trek, with a strong spring peak. These findings guide stock, staffing and marketing decisions, while RFM segmentation supports customer-specific actions. The project also shows that careful data preparation and integrity constraints protect the accuracy of downstream analysis.

5.3.2 Limitations

The analysis covers 2016–2018 and is descriptive. Refresh is manual, RFM is a snapshot of the latest order date, and different cluster counts may produce different segments.

5.3.3 Future Enhancements

Potential extensions include:

- Real-time cloud syncing. Move MySQL to a managed cloud database (such as Amazon RDS or Azure Database for MySQL) to refresh store sales continuously and improve availability, backups and scalability.
- Predictive forecasting. Long Short-Term Memory (LSTM) models could learn monthly seasonality and forecast sales by store, category and brand, helping plan stock before the spring peak.
- Automated refresh and alerts. Schedule updates and flag low stock or declining customer activity.
- Advanced customer analytics. Predict churn and customer lifetime value to refine RFM segments and personalise offers.

In conclusion, the project delivers a complete and repeatable journey from raw transactions to business insight, and it provides a foundation on which more advanced and more timely analytics can be built.